Übungsblatt 2

Abgabe bis spätestens 29.10.2026 in Stud.IP

Allgemeiner Hinweis

Für dieses Übungsblatt benötigt ihr das in StudIP zur Verfügung gestellte Archiv `ueb2-vorgabe.zip` mit Vorgabedateien.

Aufgabe 1 (2 Punkte)

In Unix-artigen Betriebssystemen werden alle Geräte wie Dateien behandelt. Dies betrifft auch den Standardeingabekanal `STDIN` und die Ausgabekanäle `STDOUT` (Standardausgabe) und `STDERR` (Fehler) eines Programms. Eine Nutzerin kann beim Aufruf eines Programms zum Beispiel entscheiden, ob die Eingabe aus einer Datei gelesen oder die Ausgabe in eine Datei geschrieben werden soll. In dieser Aufgabe gilt es, Ein- und Ausgabeumlenkungen zu üben.

Ergattert in der CTF-Plattform die Flag für die Challenge `Umleitung`. Erstellt einen oder notfalls mehrere Screenshots, auf der die **gesamte Aufgabe der Challenge** (Teilaufgaben a-f) sowie die **erhaltene Flag** zu sehen ist.

Dokumentiert Euer Vorgehen. Beschreibt für jede Zeile der Aufgabe kurz, welche Ein- und Ausgabekanäle (STDIN, STDOUT, STDERR) welcher Programme durch welchen Operator jeweils beteiligt sind. Nennt mindestens eine Quelle, die Ihr für die Bearbeitung der Aufgabe verwendet habt. Ihr dürft auch die Vorlesungsfolien als Quelle angeben. Schreibt dann aber dazu, auf welchen Seiten in welchem Foliensatz die von Euch verwendeten Informationen stehen.

Aufgabe 2 (4 Punkte)

Löst in unserer CTF-Plattform die Challenge `Prozessbaum`. Erstellt einen oder notfalls mehrere Screenshots, auf denen die Kommandos, der von Euch erstellte Prozessbaum sowie die erhaltene Flag zu sehen sind.

Beschreibt Euer Vorgehen zum Lösen der Aufgabe: wie habt Ihr herausgefunden, wie der Prozessbaum am Ende aussieht? Welche Quellen und Werkzeuge habt Ihr dafür verwendet? Nennt mindestens eine. Gebt ggf. auch die Vorlesungsfolien (mit Seitenzahl) als Quelle an.

Referenzplattformen für die Kommandos in der Challenge sind die x-Rechner und die aus unseren VM-Images erstellten virtuellen Maschinen.

Aufgabe 3 (2 Punkte)

Schreibt auf Basis des Gerüsts in `aufgabe2/sighandler.cc` ein einfaches C++-Programm mit dem Namen `sighandler`, das mittels `sigaction` einen Handler für das Signal `SIGCONT` installiert. In eurem Signalhandler soll die Nummer des empfangenen Signals ausgegeben werden sowie die Prozess-ID und die User-ID des *sendenden* Prozesses. Danach wird die Programmausführung fortgesetzt, d. h. der Signalhandler soll das Programm nicht beenden. Das Hauptprogramm besteht aus einer Endlosschleife, in der mittels `pause` auf eingehende Signale gewartet wird.

Hinweis: Benutzt in eurer Lösung die Variante `sa_sigaction` zum Setzen eures Handlers, d. h. `sa_flags` muss auf `SA_SIGINFO` gesetzt werden (siehe `man sigaction`).

Dokumentiert Eure Lösung (sowohl im Quellcode als auch ausformuliert in Eurer Abgabe). Gebt Quellen an, aus denen hervorgeht, wie Ihr auf die Lösung gekommen seid (mindestens eine Quelle). Testet **ausführlich** Eure Lösung und dokumentiert Eure Tests.

Abgabe

Bis 23:59 Uhr am 29.10.2026 digital in StudIP. **Es gelten die vereinbarten Scheinbedingungen (siehe StudIP).** Bitte beachtet unsere ergänzenden Hinweise ebenda.

Ladet die abgaberelevanten Dateien in DoIT hoch.

Eure Ansätze und der gewählte Lösungsweg müssen nachvollziehbar sein. Achtet insofern auf eine saubere Dokumentation im Quelltext und im abgegebenen PDF-Dokument. Benennt alle von Euch verwendeten Quellen, auch Zusammenarbeit mit anderen Gruppen, Vorlesungfolien (mit Seitenzahl!) sowie ggf. verwendete Unterlagen aus früheren Jahrgängen.

Programmieraufgaben sind im Zweifel in C++20 zu entwickeln. Die Korrektheit der Lösung bzw. deren Grenzen sind grundsätzlich in der Abgabe nachzuweisen. Dies geschieht neben der Dokumentation des Programmcodes in den Quelldateien und zusätzlich in dem abgegebenen PDF-Dokument mit der Lösungsbeschreibung durch geeignete Tests, deren Auswahl und Eignung begründet werden müssen. Als **Referenzplattformen** gelten die Linux-Rechner im Rechnerpool in MZH E0 und die aus unseren VM-Images erstellten virtuellen Maschinen.